

Part 1: The Phantom Bug

The program generated 50,000,000 random points and used 4 threads to approximate π .

Run 1: π = 3.141402, time = 5173.45 ms

Run 2: π = 3.141628, time = 5158.69 ms

Run 3: π = 3.141556, time = 5165.29 ms

Run 4: π = 3.141647, time = 5168.17 ms

Run 5: π = 3.141778, time = 5166.20 ms

The average value was approximately **3.141602**.

The Python implementation did not reproduce the expected Java data-race result of $\pi \approx 1.8-2.4$. This is because Python threads are affected by the Global Interpreter Lock (GIL), so the behavior differs from native Java threads.

Part 2: The Synchronization Trap

Synchronized version:

- π = **3.142025**
- Execution time = **8709.80 ms**

Single-threaded version:

- π = **3.142242**
- Execution time = **8459.49 ms**

The synchronized version produced an accurate approximation of π , but it was slower than the single-threaded version because threads repeatedly synchronized access to the shared counter. This synchronization creates overhead and prevents efficient parallel execution.

Part 3: OpenMP-Style Reduction

Threads | Runtime (ms) | Speedup | Efficiency

1	10346.61	1.00x	100.00%
2	10297.61	1.00x	50.24%
4	10303.82	1.00x	25.10%
8	10297.22	1.00x	12.56%
16	10332.37	1.00x	6.26%
32	10362.54	1.00x	3.12%

The total workload was **100,000,000 iterations**.

The results show almost no speedup when increasing the number of Python threads. The runtime remained around 10.3 seconds because Python's GIL prevents CPU-bound Python threads from executing in parallel across multiple CPU cores.

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 Threads?

The 8-core CPU did not run twice as fast with 16 threads compared with 8 threads because the CPU has only 8 logical cores. Additional threads have to share the available CPU resources, and thread management introduces overhead. In the Python implementation, the GIL also prevents CPU-bound threads from achieving true parallel execution.

2. Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized version was slower than the single-threaded version because every update of the shared counter required synchronization. Threads had to wait for access to the shared variable, creating synchronization overhead and reducing the benefit of parallel execution.